MATH3670 Project Part III
April 27, 2025

Nicholas Stone
903887158
MATH-3670-N

Preliminary information: All code referenced within the responses and within the appendix is available through the provided GitHub repository link and can be run by typing "python3 [file_name.py]" into the terminal.

I. Dataset #1: Average IQ vs. Literacy Rate

I.a. Data and Description

The restraints for Dataset #1 are $2000 > n > 100$. The provided Dataset contains 193 entries, as can be assessed by running the following with the provided file path in the Appendix:

```
print(f"Total number of entries (rows) in the dataset: {len(df_avg_iq)}")
```

Or by checking the total number of rows in the dataset, which can be found here as a .txt file in my Github repository for this project. Please utilize the earlier link to access the full dataset. A snippet of the data can also be found in the Appendix.

The source to this data can be found here. The data can be downloaded locally by running **dataset1.py** in your terminal. It will provide you with a file path, which is relative to your computer and must be utilized in **dataset1code.py** or else it will not recognize the file path.

Average IQ will be X_i, where Literacy Rate will be Y_i. As you can see throughout the scatter plot below, As a country's average IQ increases, the country's literacy rate tends to as well, indicating a general upward trajectory up to a literacy rate of 1.0, which flattens around an average IQ of 70, where most countries with IQs of 70 or above tend to maintain high literacy rates from ~.9 to 1.0. The data points which are below an IQ of 75 are much more sparse in comparison to those countries which have higher average IQs of 75.

I.b. Scatterplot of Average IQ per Country vs. Literacy Rate

Below you will find a scatterplot of the data X_i vs Y_i:

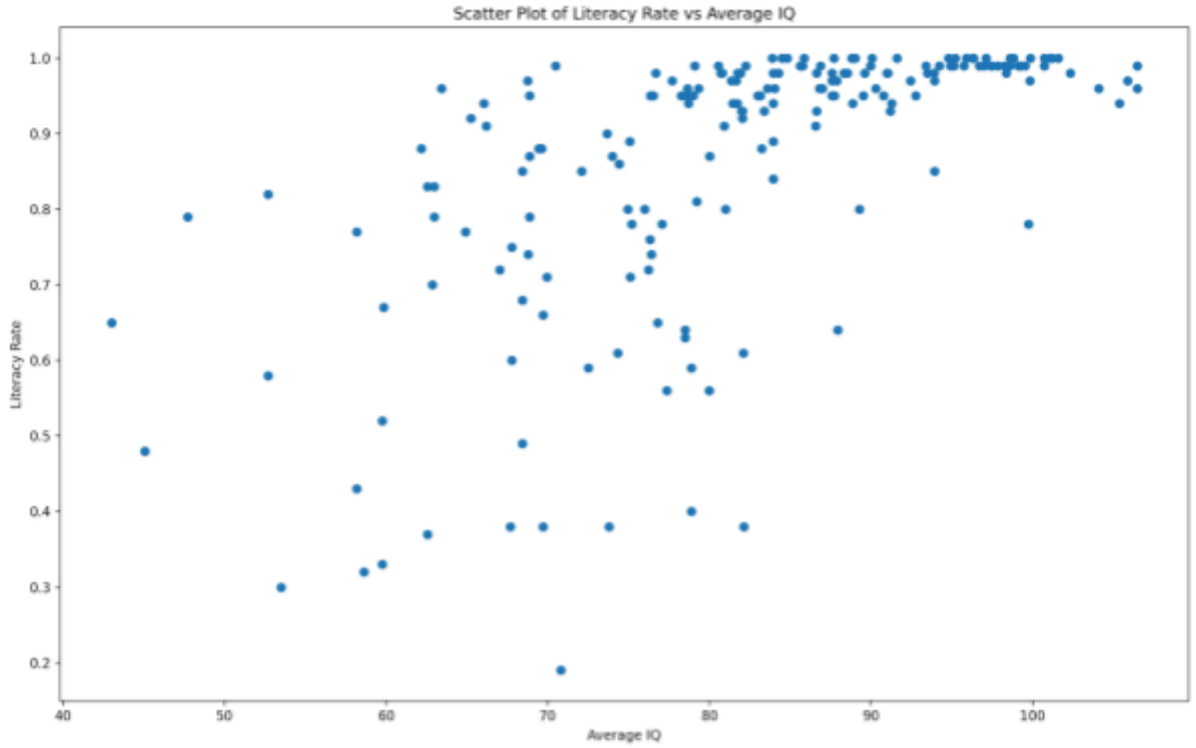


Figure 1: Scatter Plot of Y vs. X .... Part I.B

I.c. Linear Fit of Data

I.c.1.

We are tasked to Calculate the coefficients for the linear regression fit $Y_L = A + BX$ based on Dataset #1. Utilizing:

$$A = \overline{Y} - B\overline{X} \qquad B = \frac{\sum_i^N Y_i X_i - \sum_i^N Y_i \overline{X_i}}{\sum_i^N X_i^2 - N\overline{X}^2}$$

With N = our number of data points, being 193 entries, we can calculate by cd'ing into the **dataset1** folder and running **python3 scatterandtrendline.py** in the terminal. Our returned values are:

B (slope) = 0.008579473122902834
A (intercept) = 0.16032071622048139

This indicates a $Y_L = A + BX$ form of $Y_L = 0.16032071622048139 + 0.008579473122902834X$. This tracks with a visual check from the above scatter plot. If we estimate the y-intercept, it is reasonable to believe that it would land around 0.16 given the upward trajectory of the rest of the data. Additionally, a small slope does track given that the

increased literacy rate is divided by intervals of 0.1 while stretched over an IQ range of around 40 to around 110.

I.c.2.

We are tasked to now graph Y_L over the actual data, and highlight two particular values:

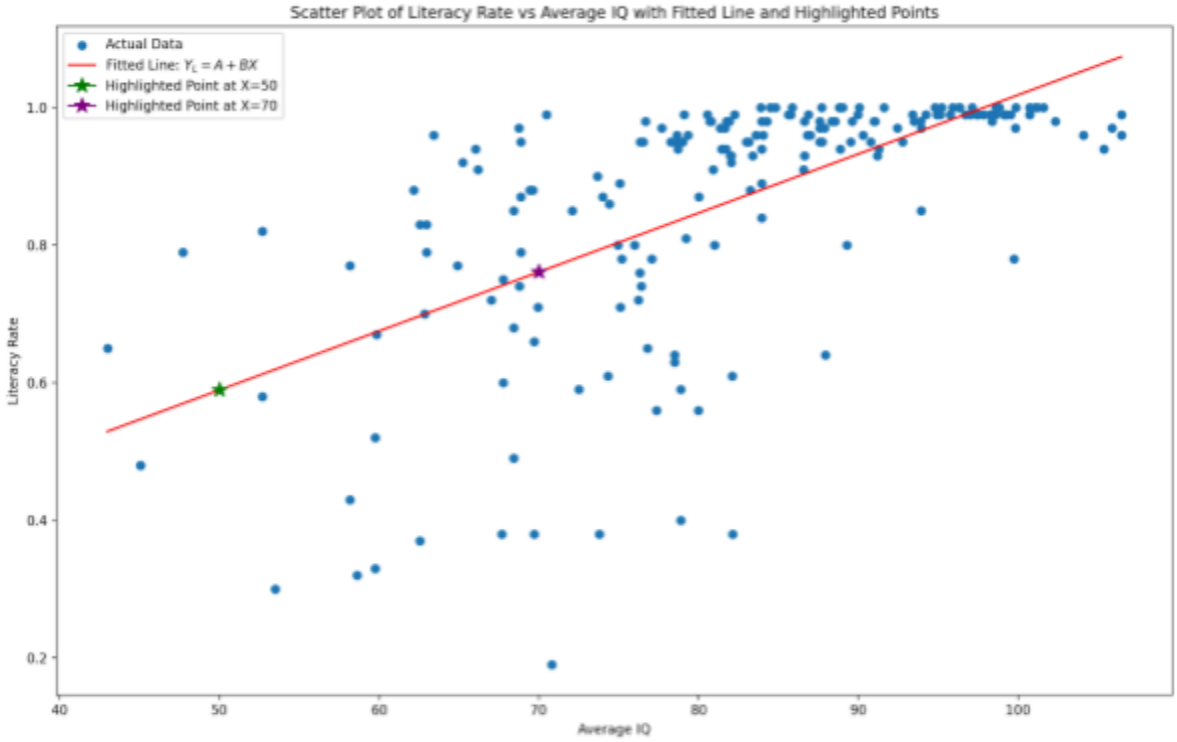


Figure 2: Linear regression + Scatter Plot of Y vs. X, part I.C.2

I.c.3.

The equations for SS_{RL} and R_L^2 are as follows:

$$SS_{RL} = \sum_{i=1}^{N}(Y_i - (A + BX_i))^2 \qquad R_L^2 = 1 - \frac{SS_{RL}}{S_{YY}} \qquad S_{YY} = \sum_{i=1}^{N}(Y_i - \overline{Y})^2$$

Additionally accounting for S_{YY}. Its code implementation can be observed within **ic3.py**. The calculated SS_{RL} was found to be *3.725402410908945*, while S_{YY} was found to be *6.238916062176164*. Consequently, R_L^2, which is 1 minus the ratio of the two, is calculated to be *0.4028766577748273*.

I.d. Exponential Fit of Data

I.d.1.

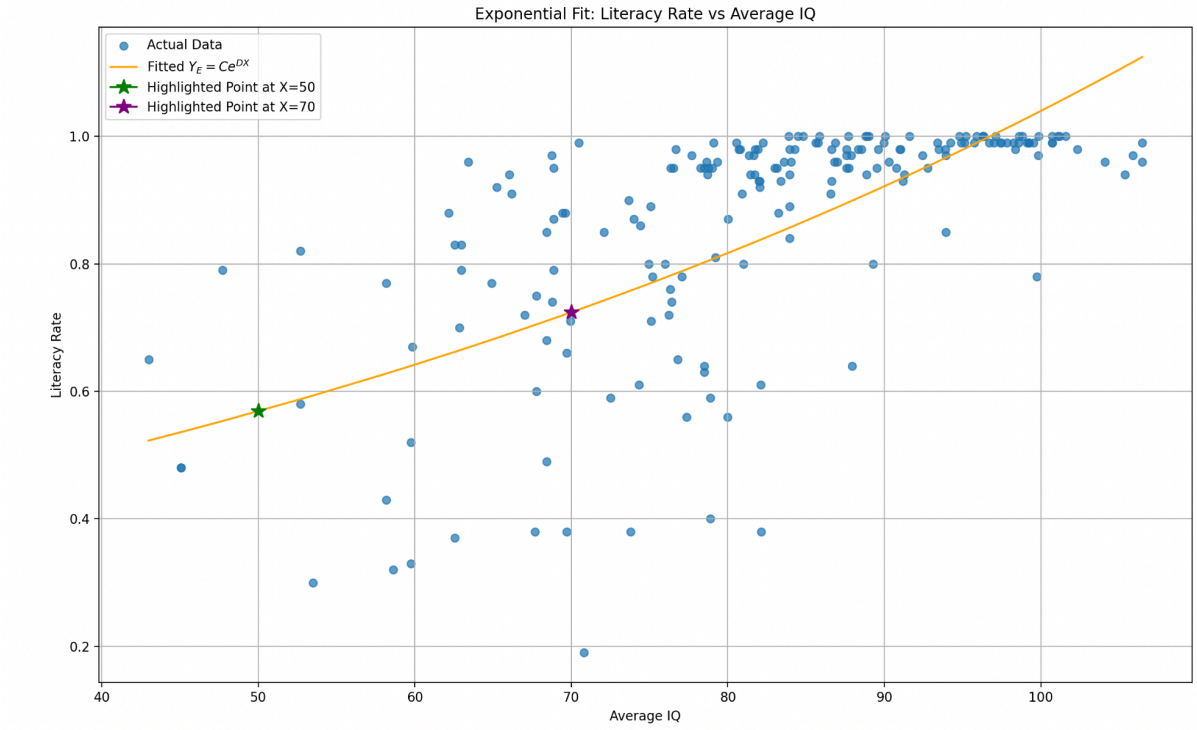
The equations for the calculation of the coefficients for the exponential fit $Y_E = Ce^{DX}$ are shown below:

$$Y_E = Ce^{DX} \qquad \ln(Y_E) = \ln(C) + DX$$

Upon calculation, which was calculated through **id1.py**, C was computed to be *0.3112906108336327* while D was found to be *0.012062344611457108*.

I.d.2.

Figure 3, the exponential regression for Y vs. X, with two highlighted values, is as shown:



I.d.3.

We are subsequently tasked to determine SS_{RE} for Y_E and R_E^2. The equations utilized are shown below:

$$SS_{RE} = \sum_{i=1}^{N}(Y_i - (C\exp(DX_i)))^2 \qquad S_{YY} = \sum_{i=1}^{N}(Y_i - \overline{Y})^2 \qquad R_E^2 = 1 - \frac{SS_{RE}}{S_{YY}}$$

Additionally accounting for S_{YY}. Its code implementation can be observed within **id3.py**. The calculated SS_{RE} was found to be *3.945879101154403*, while S_{YY} was found to be *6.238916062176164*. Consequently, R_L^2, which is 1 minus the ratio of the two, is calculated to be *0.3675377161945562*.

I.e. Discussion

Two key metrics were utilized to determine which approximation is the better of the two shown above. As the SSR measures the total squared difference between the Y values

and the model's predictions, it logically makes sense that a lower SSR will indicate a model that better suits the data. Conversely, the R^2 value measures the proportion of variance, a higher value logically will infer a better fit to the data. Since the SSR is lower and the R^2 value is greater in the linear model, we can infer that the linear model is a more consistent mode to represent the data.

Linear Model:

SS_{RL}: *3.725402410908945*

R_L^2: *0.4028766577748273*

Exponential Model:

SS_{RE}: *3.945879101154403*

R_L^2: *0.3675377161945562*

II. Dataset #1: Histogram of Age vs. Relative Frequency of having Heart Disease

II.a. Data and Description

The restraints for Dataset #2 are $5000 > n > 100$. The provided Dataset contains 270 entries, as can be assessed by checking the total number of rows in the dataset, which can be found here as a .txt file in my Github repository for this project. Please utilize the earlier link to access the full dataset. A snippet of the data can also be found in the Appendix.

The source to this data can be found here. The data can be downloaded locally by running **dataset2.py** in your terminal. It will provide you with a file path, which is relative to your computer.

What you will observe is that the x-axis represents Age, while the y-axis represents the frequency of heart disease. This is effectively showcasing the frequency of heart disease amongst decades of the people who were assessed. The graph below showcases a large concentration of heart disease being reported towards late 50's and early 60's, while it tapers off towards younger ages and even drops off towards older ages. The initial frequency histogram shows that the height of cases per age group is approaching 60, and the relative frequency histogram helps put that into perspective by denoting that as a little over .200.

II.b. Sample Mean and Variance

II.b.1.

The equation for the sample mean is shown below:

$$\overline{X} = \frac{1}{N_X} \sum_{i=1}^{N_X} X_i$$

It's code implementation can be found in **iib.py**, and the sample mean was calculated to be *54.43333333333333*, indicating that the sample mean of Dataset #2 is found to be around 54.43 years.

II.b.2.

The equation for the sample variance is shown below:

$$S_X^2 = \frac{1}{N_X - 1} \sum_{i=1}^{N_X} (X_i - \overline{X})^2.$$

It's code implementation can be found in **iib.py**, and the sample variance was calculated to be *82.97509293680297*, indicating that the spread of Dataset #2 is found to be around 82.98 years.

II.c. Relative Frequency Histogram and Quartiles

II.c.1.

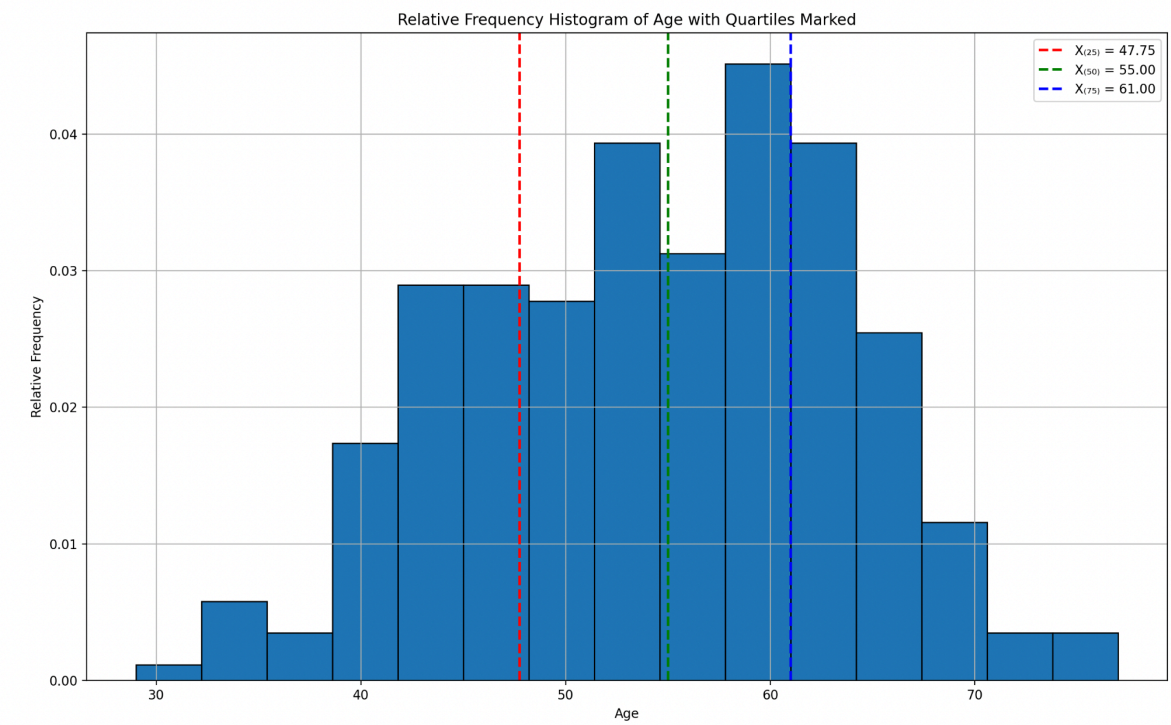
We are now asked to provide three values which separate the four quartiles of the data, X_{25}, X_{50}, X_{75}. The following three values separate the four quartiles of the data:

Number of data points (N_X) = 270
X_{25} = 47.75
X_{50} (median) = 55.0
X_{75} = 61.0

Its code implementation can be observed in **iic1.py**.

II.c.2.

Figure 4, the relative frequency diagram with the values of the quartiles clearly marked is shown below. This graph can also be attained by running **iic2.py**.



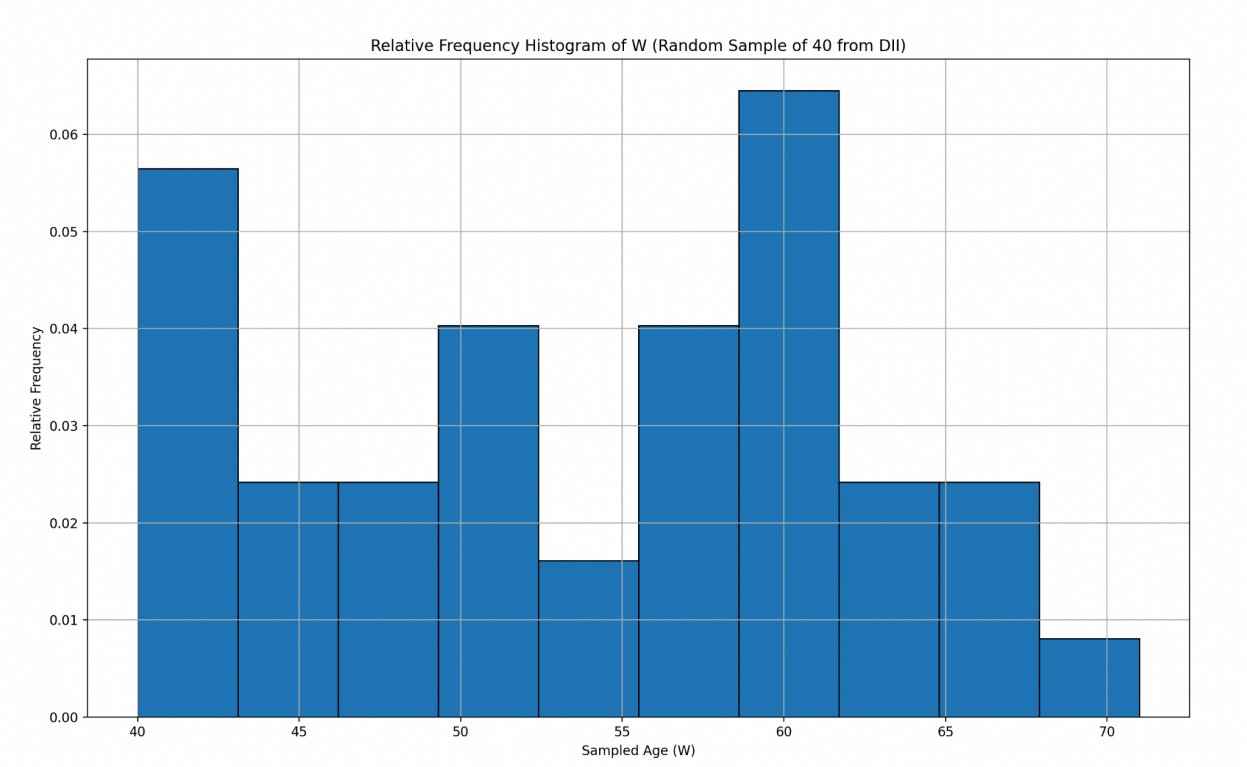
II.d. Comparison of Random Subsample

II.d.1.

A new dataset W was randomly created within the available values of Dataset #2. 40 randomly selected values are visualized in the below relative frequency histogram, and can be observed in its coding implementation through **iid1.py**.

II.d.2.

Below is Figure 5, the relative frequency histogram for W. Its coding implementation can be observed in **iid1.py**.



The sample mean of W was calculated to be *53.825*, and in utilizing the following equation:

For $\alpha = .05$:

$$\overline{W} - z_{\alpha/2} S_X/\sqrt{(40)} < \mu < \overline{W} + z_{\alpha/2} S_X/\sqrt{(40)} \quad z_{\alpha/2} = 1.96$$

We were able to calculate a 95% confidence interval, *(51.00207103779593, 56.64792896220408)*. Comparing with the sample mean of X, which was previously found to be *54.43333333333333*, we find that W's sample mean is smaller and X's sample mean, in fact, falls between the confidence interval calculated. The coding implementation of these calculations can be observed in **iid2.py**.

II.d.3.

We will attempt to find the p-value from the hypothesis:

$$H_0 : \mu = \overline{X}$$

To determine the p-value, we will employ a standard normal (Z) statistic since the population variance is known, the sample size is significantly large, and we are testing the population mean against the sample mean of X. Therefore, our test statistic formula is as follows:

$$Z = \frac{\overline{W} - \overline{X}}{S_X / \sqrt{n}}$$

This follows the standard normal distribution under H_O.

With a calculated Z-statistic of -0.4223745440630492, the p-value is calculated according to the below equation (Notice that it is two-tailed, denoted by being multiplied by 2):

$$p\text{-value} = 2 \cdot P(Z > |z|)$$

The p-value was subsequently computed to be 0.672751655414368, which means we fail to reject H_O at the 5% significance level since the p-value is much greater than alpha = 0.05.

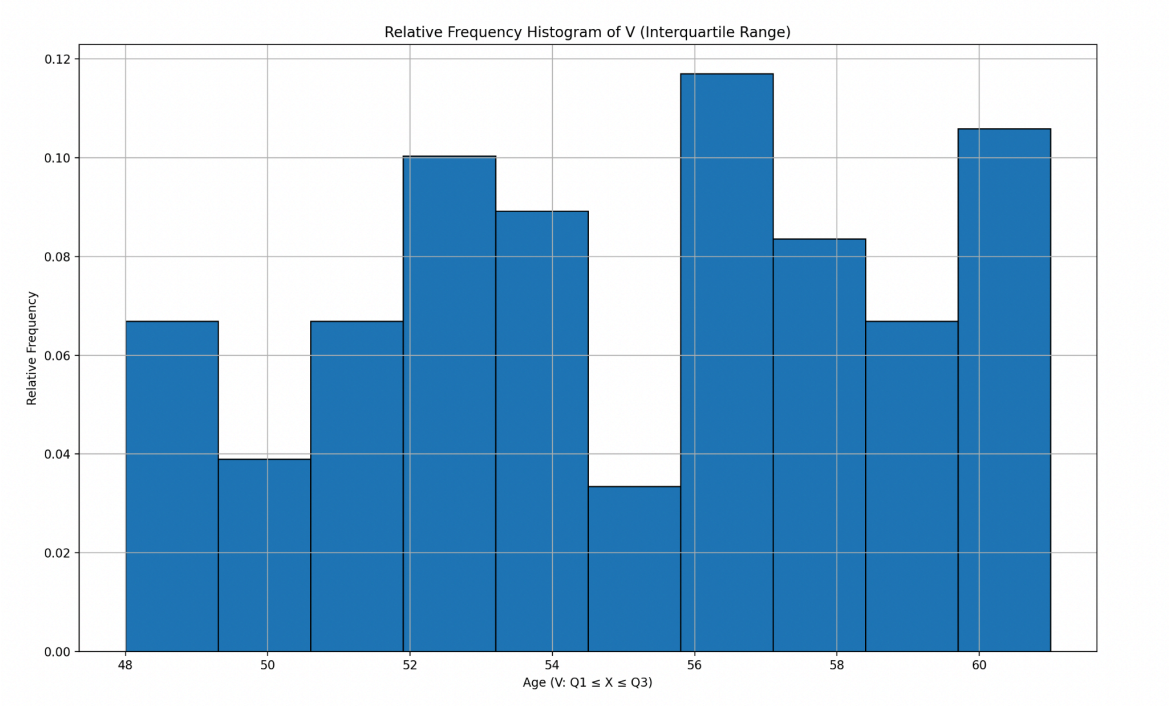
II.e. Comparison of Non-Random Subsample

II.e.1.

We are now to create a new dataset V from Dataset #2 consisting of all values from the second and third quartiles. There are exactly 138 values within those parameters.

II.e.2.

The relative histogram for V is provided below. The code implementation can be accessed through **iie1.py**.



II.e.3.

We will attempt to find the p-value from the hypothesis:

To determine the p-value, we will again employ a standard normal (Z) statistic since the population variance is known and N_V is sufficiently large. The formulas, once again, remain:

$$Z = \frac{\overline{V} - \overline{X}}{S_X / \sqrt{N_V}} \quad \text{then} \quad p = 2 \cdot P(Z > |z|)$$

In which a Z-statistic of *0.7588267152980456*, and a p-value of *0.44795622233462096* were both calculated.

II.e.4.

We are now tasked to calculate the sample mean, the sample variance, and the p-value for the dataset V, assuming that the actual variance for V is unknown, and maintaining our hypothesis that:

$$H_0 : \mu = \overline{X}$$

We will now proceed with a t-statistic, given a moderate N_V and an unknown population standard deviation. Therefore, our formula we will utilize is as follows:

$$t = \frac{\overline{V} - \overline{X}}{S_V / \sqrt{N_V}}$$

Our degrees of freedom are N_V - 1, which would be 137. We will proceed with a two-tailed T-test:

$$p = 2 \cdot P(T > |t|), \quad \text{where } T \sim t_{df}$$

Upon calculations which are detailed in **iie4.py**, we find a t-statistic of *1.8233124776846477*, and a p-value of *0.07043598183841393*, which indicates that we fail to reject H_0 at the 5% level.

II.f. Discussion

The histogram for X_i show the full spread of the data, including both of the tails, and because it has the most values out of all three of the histograms, it logically makes sense that it is the most variable and closest to a normal distribution, while still showing a minor left skew. The histogram of W_i contained a random sample of 40 values from X and represents a snapshot of the entire distribution. Because it is randomly sampled, it embodies some of the behavior of X, however, it contains a lot of sampling noise which, in the case of the random sample utilized, actually shows a bit of a right skew. Finally, the histogram of V_i contains only values from X_{25} and X_{75} which excludes the most extreme values. This, by nature, makes the histogram more symmetric and narrower, with much less variability, though still containing characteristics of the X distribution.

For W_i with a known variance, a normal distribution Z-test was utilized. By definition, the assumption of a known population variance infers a smaller standard error. For V_i with a known variance, we also employed a Z-test, however, because of the lack of tail data, the mean was likely altered and resulted in a different Z-score and p-value. Finally, for V_i with an unknown variance, we instead utilized a t-test, and because we have to estimate variance, this infers a wider distribution and a subsequent larger p-value.

By limiting our data to the two central quartiles, we risk reducing the variance by suggesting that the spread of the data is much more stable and tight to the middle 50%, when in reality, it could be more spread out. On that note, we could miss crucial outliers, which could drastically impact the data or provide further insight to it. In restricting our data, we are effectively biasing it, and making it a misleading inference to the situation we are attempting to assess.

V. Code and Documentation

The code and associated documentation can be found here. This link to my Github repository contains the below file structure and project history for your convenience. Else, code is provided in the appendix below.

```
/project_part3
├── /dataset1
├── /dataset2
└──/images
```

Appendix

Reference 1: dataset.py

```python
#Taken from Kaggle, run file to download data locally

import kagglehub

path = kagglehub.dataset_download("mlippo/average-global-iq-per-country-with-other-stats")

print("Path to dataset files:", path)
```

Reference 2: ic3.py

```python
import pandas as pd
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/mlippo/average-global-iq-per-country-with-other-stats/versions/3/avgIQpercountry.csv"
data = pd.read_csv(file_path)

# Clean data
data.columns = data.columns.str.strip()
data = data.dropna(subset=['Average IQ', 'Literacy Rate'])

# Determine variables
X = data['Average IQ']
Y = data['Literacy Rate']

# Calculate terms
N = len(X)
X_mean = X.mean()
Y_mean = Y.mean()
sum_YiXi = (Y * X).sum()
sum_Xi2 = (X * X).sum()

# A and B calculations
B = (sum_YiXi - N * Y_mean * X_mean) / (sum_Xi2 - N * X_mean**2)
A = Y_mean - B * X_mean

print(f"Calculated coefficients:")
print(f"B (slope) = {B}")
print(f"A (intercept) = {A}")

Y_L = A + B * X

# SS_RL
SS_RL = ((Y - Y_L) ** 2).sum()

# S_YY
S_YY = ((Y - Y_mean) ** 2).sum()

# R_L^2
R_squared = 1 - (SS_RL / S_YY)

print(f"\nSS_RL = {SS_RL}")
print(f"S_YY = {S_YY}")
print(f"R_L^2 = {R_squared}")
```

Reference 3: id1.py

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/mlippo/average-global-iq-per-country-with-other-stats/versions/3/avgIQpercountry.csv"
data = pd.read_csv(file_path)

# Clean data
data.columns = data.columns.str.strip()
data = data.dropna(subset=['Average IQ', 'Literacy Rate'])

# Determine variables
X = data['Average IQ']
Y = data['Literacy Rate']

# Y-Shift
if (Y <= 0).any():
    K = abs(Y.min()) + 1
    print(f"Shifting Y by +{K} to make all values positive for log.")
    Y_shifted = Y + K
    shifted = True
else:
    Y_shifted = Y
    K = 0
    shifted = False

# Natural Log
log_Y = np.log(Y_shifted)

# Linear Regression
N = len(X)
X_mean = X.mean()
logY_mean = log_Y.mean()
sum_logYXi = (log_Y * X).sum()
sum_X2 = (X * X).sum()

D = (sum_logYXi - N * logY_mean * X_mean) / (sum_X2 - N * X_mean**2)
ln_C = logY_mean - D * X_mean
C = np.exp(ln_C)

print(f"\nExponential fit coefficients:")
print(f"D = {D}")
print(f"C = {C}")
```

```
# Two highlighted values
X1 = 50
X2 = 70

# Calculate Y_E at X1 and X2
YE1 = C * np.exp(D * X1)
YE2 = C * np.exp(D * X2)
if shifted:
    YE1 -= K
    YE2 -= K

# Values plotted on figure
X_sorted = np.sort(X)
YE_curve = C * np.exp(D * X_sorted)
if shifted:
    YE_curve -= K

# Scatterplot data
plt.scatter(X, Y, label='Actual Data', alpha=0.7)

# Exponential curve plotted
plt.plot(X_sorted, YE_curve, color='orange', label=r'Fitted $Y_E = Ce^{DX}$')

# Two highlighted points plotted
plt.plot(X1, YE1, marker='*', markersize=12, color='green', label=f'Highlighted Point at X={X1}')
plt.plot(X2, YE2, marker='*', markersize=12, color='purple', label=f'Highlighted Point at X={X2}')

# Labeling
plt.xlabel('Average IQ')
plt.ylabel('Literacy Rate')
plt.title('Exponential Fit: Literacy Rate vs Average IQ')
plt.legend()
plt.grid(True)
plt.show()
```

Reference 4: id3.py

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/mlippo/average-global-iq-per-country-with-other-stats/versions/3/avgIQpercountry.csv"
data = pd.read_csv(file_path)

# Clean data
data.columns = data.columns.str.strip()
data = data.dropna(subset=['Average IQ', 'Literacy Rate'])

# Determine variables
X = data['Average IQ']
Y = data['Literacy Rate']

# Y Shift
if (Y <= 0).any():
    K = abs(Y.min()) + 1
    print(f"Shifting Y by +{K} to make all values positive for log.")
    Y_shifted = Y + K
    shifted = True
else:
    Y_shifted = Y
    K = 0
    shifted = False

# Natural Log
log_Y = np.log(Y_shifted)

# Linear Regression
N = len(X)
X_mean = X.mean()
logY_mean = log_Y.mean()
sum_logYXi = (log_Y * X).sum()
sum_X2 = (X * X).sum()

D = (sum_logYXi - N * logY_mean * X_mean) / (sum_X2 - N * X_mean**2)
ln_C = logY_mean - D * X_mean
C = np.exp(ln_C)

print(f"\nExponential fit coefficients:")
print(f"D = {D}")
print(f"C = {C}")
```

```python
YE = C * np.exp(D * X)
if shifted:
    YE -= K

# SS_RE
SS_RE = ((Y - YE) ** 2).sum()

# S_YY
S_YY = ((Y - Y.mean()) ** 2).sum()

# R_E^2
R_squared_E = 1 - (SS_RE / S_YY)

# Results printed to terminal
print(f"\nSum of Squares of Residuals (SS_RE) = {SS_RE}")
print(f"Total Sum of Squares (S_YY) = {S_YY}")
print(f"Coefficient of Determination (R_E^2) = {R_squared_E}")
```

Reference 5: scatterandtrendline.py

```python
import pandas as pd
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/mlippo/average-global-iq-per-country-with-other-stats/versions/3/avgIQpercountry.csv"
data = pd.read_csv(file_path)

# Clean data
data.columns = data.columns.str.strip()
data = data.dropna(subset=['Average IQ', 'Literacy Rate'])

# Determine variables
X = data['Average IQ']
Y = data['Literacy Rate']

# Calculate terms
N = len(X)
X_mean = X.mean()
Y_mean = Y.mean()
sum_YiXi = (Y * X).sum()
sum_Xi2 = (X * X).sum()

# A and B calculations
B = (sum_YiXi - N * Y_mean * X_mean) / (sum_Xi2 - N * X_mean**2)
A = Y_mean - B * X_mean

print(f"Calculated coefficients:")
print(f"B (slope) = {B}")
print(f"A (intercept) = {A}")

# Y_L for each X calculation
Y_L = A + B * X

# Scatterplot plotted
plt.scatter(X, Y, label='Actual Data')

# Regression line plotted
plt.plot(X, Y_L, color='red', label='Fitted Line: $Y_L = A + BX$')

# Two chosen values
X1 = 50
X2 = 70
```

```python
# Y_L1, Y_L2 on the fitted line
YL1 = A + B * X1
YL2 = A + B * X2

# Plot the two highlighted points
plt.plot(X1, YL1, marker='*', markersize=12, color='green', label=f'Highlighted Point at X={X1}')
plt.plot(X2, YL2, marker='*', markersize=12, color='purple', label=f'Highlighted Point at X={X2}')

# Labels
plt.xlabel('Average IQ')
plt.ylabel('Literacy Rate')
plt.title('Scatter Plot of Literacy Rate vs Average IQ with Fitted Line and Highlighted Points')
plt.legend()
plt.show()
```

Reference 6: scatterplot.py

```python
import pandas as pd
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/mlippo/average-global-iq-per-country-with-other-stats/versions/3/avgIQpercountry.csv"
data = pd.read_csv(file_path)

# Clean data
data.columns = data.columns.str.strip()
data = data.dropna(subset=['Average IQ', 'Literacy Rate'])

# Determine variables
X = data['Average IQ']
Y = data['Literacy Rate']

# Scatterplot plotted
plt.scatter(X, Y)

# Labels
plt.xlabel('Average IQ')
plt.ylabel('Literacy Rate')
plt.title('Scatter Plot of Literacy Rate vs Average IQ')
plt.show()
```

Reference 7: dataset2.py

```python
#Taken from Kaggle, run file to download data locally

import kagglehub

path = kagglehub.dataset_download("luvharishkhati/heart-disease-patients-details")

print("Path to dataset files:", path)

heart_disease.csv
```

Reference 8: iib.py

```python
import pandas as pd

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variables
X = df["age"]

# # of observations
N_X = len(X)

# Sample Mean calculation
sample_mean = sum(X) / N_X

# Sample Variance calculation
sum_squared_diff = sum((xi - sample_mean) ** 2 for xi in X)
sample_variance = sum_squared_diff / (N_X - 1)

# Display
print(f"Sample Mean (X̄) for 'age': {sample_mean}")
print(f"Sample Variance (S²_X) for 'age': {sample_variance}")
```

Reference 9: iic1.py

```python
import pandas as pd

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variables
X = df["age"]
X_sorted = sorted(X)
N_X = len(X_sorted)

# Calculate quartiles
pos_25 = 0.25 * (N_X + 1)
pos_50 = 0.50 * (N_X + 1)
pos_75 = 0.75 * (N_X + 1)
def interpolate(sorted_list, position):
    lower_index = int(position) - 1
    upper_index = lower_index + 1
    fraction = position - int(position)

    if upper_index >= len(sorted_list):
        return sorted_list[lower_index]
    else:
        return sorted_list[lower_index] + fraction * (sorted_list[upper_index] - sorted_list[lower_index])

# Fetch quartiles
X_25 = interpolate(X_sorted, pos_25)
X_50 = interpolate(X_sorted, pos_50)
X_75 = interpolate(X_sorted, pos_75)

# Display quartiles
print(f"Number of data points (N_X) = {N_X}")
print(f"X_25 (25th percentile) = {X_25}")
print(f"X_50 (50th percentile, median) = {X_50}")
print(f"X_75 (75th percentile) = {X_75}")
```

Reference 10: iic2.py

```python
import pandas as pd
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variable
X = df["age"]
X_sorted = sorted(X)
N_X = len(X_sorted)

# Calculate quartiles
pos_25 = 0.25 * (N_X + 1)
pos_50 = 0.50 * (N_X + 1)
pos_75 = 0.75 * (N_X + 1)
def interpolate(sorted_list, position):
    lower_index = int(position) - 1
    upper_index = lower_index + 1
    fraction = position - int(position)
    if upper_index >= len(sorted_list):
        return sorted_list[lower_index]
    else:
        return sorted_list[lower_index] + fraction * (sorted_list[upper_index] - sorted_list[lower_index])
X_25 = interpolate(X_sorted, pos_25)
X_50 = interpolate(X_sorted, pos_50)
X_75 = interpolate(X_sorted, pos_75)

# Create Relative Frequency Histogram
plt.figure(figsize=(10, 6))
plt.hist(X, bins=15, edgecolor='black', density=True)

# Plot quartiles
plt.axvline(X_25, color='red', linestyle='dashed', linewidth=2, label=f'X₍25₎ = {X_25:.2f}')
plt.axvline(X_50, color='green', linestyle='dashed', linewidth=2, label=f'X₍50₎ = {X_50:.2f}')
plt.axvline(X_75, color='blue', linestyle='dashed', linewidth=2, label=f'X₍75₎ = {X_75:.2f}')

# Labels
plt.xlabel('Age')
plt.ylabel('Relative Frequency')
plt.title('Relative Frequency Histogram of Age with Quartiles')
plt.legend()
plt.grid(True)
plt.show()
```

```python
# Printed values
print(f"Number of data points (N_X) = {N_X}")
print(f"X_25 (25th percentile) = {X_25}")
print(f"X_50 (50th percentile, median) = {X_50}")
print(f"X_75 (75th percentile) = {X_75}")
```

Reference 11: iid1.py

```python
import pandas as pd
import matplotlib.pyplot as plt
import random
random.seed(42)

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variable
DII = df["age"].tolist()

# Random sample of 40
W = random.sample(DII, 40)

# Plot W, the relative frequency histogram
plt.figure(figsize=(10, 6))
plt.hist(W, bins=10, edgecolor='black', density=True)

# Labels
plt.xlabel("Sampled Age (W)")
plt.ylabel("Relative Frequency")
plt.title("Relative Frequency Histogram of W (Random Sample of 40 from DII)")
plt.grid(True)
plt.show()

# Printed values
print("Random Sample (W):")
print(W)
```

Reference 12: iid2.py

```python
import pandas as pd
import matplotlib.pyplot as plt
import random
import math
from scipy.stats import norm
random.seed(42)

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine Variables
X = df["age"].tolist()

# Sample mean and variance computations
N_X = len(X)
X_mean = sum(X) / N_X
S_X2 = sum((xi - X_mean)**2 for xi in X) / (N_X - 1)
S_X = math.sqrt(S_X2)

# Create W, 40 random samples
W = random.sample(X, 40)
W_mean = sum(W) / len(W)

# 95% Confidence interval
z = 1.96
margin_error = z * S_X / math.sqrt(len(W))
CI_lower = W_mean - margin_error
CI_upper = W_mean + margin_error

# Print CI
print("=== Confidence Interval ===")
print(f"Sample mean of W (W̄): {W_mean}")
print(f"Sample mean of X (X̄): {X_mean}")
print(f"Sample standard deviation of X (S_X): {S_X}")
print(f"95% Confidence Interval for μ: ({CI_lower}, {CI_upper})")
print("Is X̄ within the CI?", CI_lower <= X_mean <= CI_upper)

# Plot W histogram
plt.figure(figsize=(10, 6))
plt.hist(W, bins=10, edgecolor='black', density=True)
plt.xlabel("Age (Sampled W)")
plt.ylabel("Relative Frequency")
plt.title("Relative Frequency Histogram of W (n=40)")
plt.grid(True)
plt.show()
```

```python
Z_stat = (W_mean - X_mean) / (S_X / math.sqrt(len(W)))

# Two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(Z_stat)))

# Print hypothesis test results
print("\n=== Hypothesis Test ===")
print(f"Z statistic: {Z_stat}")
print(f"p-value: {p_value}")

# Deliberation
if p_value < 0.05:
    print("Reject H₀ at the 5% significance level.")
else:
    print("Fail to reject H₀ at the 5% significance level.")
```

Reference 13: iie1.py

```python
import pandas as pd
import matplotlib.pyplot as plt

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variables
X = df["age"]
X_sorted = sorted(X)
N_X = len(X_sorted)
def interpolate(sorted_list, position):
    lower_index = int(position) - 1
    upper_index = lower_index + 1
    fraction = position - int(position)
    if upper_index >= len(sorted_list):
        return sorted_list[lower_index]
    return sorted_list[lower_index] + fraction * (sorted_list[upper_index] - sorted_list[lower_index])

# X25 and X75, # of variables
pos_25 = 0.25 * (N_X + 1)
pos_75 = 0.75 * (N_X + 1)
X_25 = interpolate(X_sorted, pos_25)
X_75 = interpolate(X_sorted, pos_75)
V = [x for x in X if X_25 <= x <= X_75]
N_V = len(V)

# V, relative frequency histogram
plt.figure(figsize=(10, 6))
plt.hist(V, bins=10, edgecolor='black', density=True)
plt.xlabel("Age (V: Q1 ≤ X ≤ Q3)")
plt.ylabel("Relative Frequency")
plt.title("Relative Frequency Histogram of V (Interquartile Range)")
plt.grid(True)
plt.show()

# Printed values
print(f"X_25 (Q1): {X_25}")
print(f"X_75 (Q3): {X_75}")
print(f"Number of values in V (N_V): {N_V}")
```

Reference 14: iie3.py

```python
import pandas as pd
import math
from scipy.stats import norm

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variables
X = df["age"].tolist()
X_sorted = sorted(X)
N_X = len(X_sorted)
def interpolate(sorted_list, position):
    lower_index = int(position) - 1
    upper_index = lower_index + 1
    fraction = position - int(position)
    if upper_index >= len(sorted_list):
        return sorted_list[lower_index]
    return sorted_list[lower_index] + fraction * (sorted_list[upper_index] - sorted_list[lower_index])

# Q1 and Q3 computation
pos_25 = 0.25 * (N_X + 1)
pos_75 = 0.75 * (N_X + 1)
X_25 = interpolate(X_sorted, pos_25)
X_75 = interpolate(X_sorted, pos_75)
V = [x for x in X if X_25 <= x <= X_75]
N_V = len(V)
V_mean = sum(V) / N_V

# Stats
X_mean = sum(X) / N_X
S_X2 = sum((xi - X_mean)**2 for xi in X) / (N_X - 1)
S_X = math.sqrt(S_X2)

# Z-statistic computation
Z_stat = (V_mean - X_mean) / (S_X / math.sqrt(N_V))

# Two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(Z_stat)))
```

```python
# Print results
print("=== Hypothesis Test for V ===")
print(f"X̄ (population mean under H₀): {X_mean}")
print(f"V̄ (sample mean of V): {V_mean}")
print(f"N_V (sample size of V): {N_V}")
print(f"Z statistic: {Z_stat}")
print(f"p-value: {p_value}")

# Deliberation
if p_value < 0.05:
    print("Reject H₀ at the 5% level.")
else:
    print("Fail to reject H₀ at the 5% level.")
```

Reference 15: iie4.py

```python
import pandas as pd
import math
from scipy.stats import t

# File path
file_path = "/Users/nicholasstone/.cache/kagglehub/datasets/luvharishkhati/heart-disease-patients-details/versions/1/heart_disease.csv"
df = pd.read_csv(file_path)

# Determine variables
X = df["age"].tolist()
X_sorted = sorted(X)
N_X = len(X_sorted)
def interpolate(sorted_list, position):
    lower_index = int(position) - 1
    upper_index = lower_index + 1
    fraction = position - int(position)
    if upper_index >= len(sorted_list):
        return sorted_list[lower_index]
    return sorted_list[lower_index] + fraction * (sorted_list[upper_index] - sorted_list[lower_index])

# Q1 and Q3 computation
pos_25 = 0.25 * (N_X + 1)
pos_75 = 0.75 * (N_X + 1)
X_25 = interpolate(X_sorted, pos_25)
X_75 = interpolate(X_sorted, pos_75)
V = [x for x in X if X_25 <= x <= X_75]
N_V = len(V)
V_mean = sum(V) / N_V
X_mean = sum(X) / N_X

# Sample variance and std dev
S_V2 = sum((vi - V_mean)**2 for vi in V) / (N_V - 1)
S_V = math.sqrt(S_V2)

# t-statistic computation
t_stat = (V_mean - X_mean) / (S_V / math.sqrt(N_V))
df = N_V - 1

# Two-tailed p-value
p_value = 2 * (1 - t.cdf(abs(t_stat), df))
```

```python
# Output results
print("=== Hypothesis Test for V (Unknown Variance) ===")
print(f"V̄ (sample mean of V): {V_mean}")
print(f"X̄ (reference mean under H₀): {X_mean}")
print(f"N_V = {N_V}, Degrees of freedom = {df}")
print(f"Sample std dev of V (S_V): {S_V}")
print(f"t-statistic: {t_stat}")
print(f"p-value: {p_value}")

# Deliberation
if p_value < 0.05:
    print("Reject H₀ at the 5% level.")
else:
    print("Fail to reject H₀ at the 5% level.")
```